

Q59. Why is String Immutable in Java?

Simple Answer

A **String** in Java is immutable, which means once a String object is created, its value cannot be changed.

Any modification creates a new String object instead of changing the existing one.

Detailed Explanation

When a String is created:

```
String name = "Java";
```

The object remains unchanged throughout its lifetime.

If we write:

```
name = name + " Programming";
```

Java creates a **new String object**.

The original object "Java" remains unchanged.

Why did Java make String immutable?

- Security
- Thread Safety
- String Pool Optimization
- Better Performance
- Reliable HashMap Keys

Real-Time Example

A banking application stores:

```
Account Number
```

If Strings were mutable, malicious code could modify account numbers after validation.

Immutability prevents such security issues.

Code Example

```
public class Main {  
    public static void main(String[] args) {  
        String s1 = "Java";  
        s1.concat(" Programming");  
        System.out.println(s1);  
        s1 = s1.concat(" Programming");  
        System.out.println(s1);  
    }  
}
```

Output

```
Java  
Java Programming
```

Interview Tip

This is one of the **Top 5 Java interview questions**.

Always mention:

- Security
- Thread Safety
- String Pool
- HashMap

Key Points

- ✓ String objects are immutable.
- ✓ Modification creates a new object.
- ✓ Improves security.
- ✓ Enables String Pool optimization.

Q60. What is the String Pool?

Simple Answer

The **String Pool** is a special memory area inside the JVM where String literals are stored.

If the same String literal already exists, Java reuses the existing object instead of creating a new one.

Detailed Explanation

Example:

```
String s1 = "Java";  
String s2 = "Java";
```

Both references point to the same object.

But:

```
String s3 = new String("Java");
```

creates a new object in Heap memory.

JVM Representation

```
String Pool  
-----  
"Java"
```

```
Heap  
-----  
new String("Java")
```

Real-Time Example

Suppose an application stores the status:

```
SUCCESS  
FAILED  
PENDING
```

Thousands of requests reuse these values.

The String Pool avoids creating duplicate String objects.

Code Example

```
String s1 = "Java";  
String s2 = "Java";  
System.out.println(s1 == s2);
```

Output

```
true
```

Interview Tip

Common interview question:

Where is the String Pool located?

Answer:

It is stored in the **Heap** (since Java 7).

Key Points

- ✓ Stores String literals.
- ✓ Avoids duplicate objects.
- ✓ Saves memory.
- ✓ Improves performance.

Q61. What is the difference between String, StringBuilder, and StringBuffer?

Simple Answer

All three classes represent character sequences, but they differ in mutability and thread safety.

Feature	String	StringBuilder	StringBuffer
Mutable	✗ No	✓ Yes	✓ Yes
Thread Safe	✓ Yes (Immutable)	✗ No	✓ Yes
Performance	Slower for repeated modifications	Fastest	Slower than StringBuilder

Detailed Explanation

String

Immutable.

Every modification creates a new object.

StringBuilder

Mutable.

Best choice for single-threaded applications.

StringBuffer

Mutable.

Thread-safe because its methods are synchronized.

Real-Time Example

Suppose a report generator appends thousands of records.

Using:

```
String
```

creates thousands of unnecessary objects.

Instead:

```
StringBuilder
```

provides much better performance.

Code Example

```
StringBuilder builder = new StringBuilder();  
builder.append("Java");  
builder.append(" Spring");  
System.out.println(builder);
```

Output

Java Spring

Interview Tip

One of the most common interview questions:

Which is faster?

Answer:

StringBuilder

because it is mutable and not synchronized.

Key Points

- ✓ String → Immutable
- ✓ StringBuilder → Mutable, Fast
- ✓ StringBuffer → Mutable, Thread Safe

Q62. What is the difference between `==` and `equals()` for Strings?

Simple Answer

For Strings:

- `==` compares object references (memory addresses).
- `equals()` compares the actual content of the String.

Detailed Explanation

Example:

```
String s1 = new String("Java");
```

```
String s2 = new String("Java");
```

```
s1 == s2
```

returns:

```
false
```

because they are different objects.

But:

```
s1.equals(s2)
```

returns:

```
true
```

because both contain the same text.

Real-Time Example

Suppose two users enter:

```
Murali
```

Although the two String objects are different, their contents are the same.

The application should compare them using `equals()`.

Code Example

```
String s1 = new String("Java");  
String s2 = new String("Java");  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Output

```
false  
true
```

Interview Tip

A common mistake:

Never compare Strings using:

```
==
```

Always use:

```
equals()
```

when comparing String contents.

Key Points

- ✓ `==` compares references.
- ✓ `equals()` compares contents.
- ✓ Use `equals()` for business logic.
- ✓ One of the most frequently asked Java interview questions.

Q63. What is the difference between `String.concat()` and the `+` operator?

Simple Answer

Both `String.concat()` and the `+` operator are used to combine Strings.

- `concat()` appends one String to another.
- The `+` operator concatenates Strings and can also combine Strings with other data types by converting them to Strings.

Both create a **new String object** because Strings are immutable.

Detailed Explanation

Using `concat()`

Works only with String objects.

```
String s = "Java";  
s = s.concat(" Programming");
```

Using `+`

More flexible.

It automatically converts numbers, characters, and other objects into Strings.

```
String name = "Murali";  
int age = 30;  
  
System.out.println(name + " " + age);
```

Real-Time Example

Suppose an invoice application generates:

```
Invoice No : 101  
Customer : Murali
```

The `+` operator is commonly used to combine text and numbers.

Code Example

```
public class Main {
```

```
public static void main(String[] args) {  
    String s1 = "Java";  
    String s2 = s1.concat(" Spring");  
    String s3 = s1 + " Spring";  
    System.out.println(s2);  
    System.out.println(s3);  
}
```

Output

```
Java Spring  
Java Spring
```

Interview Tip

A common interview question:

Which is preferred?

- For a few concatenations → + is perfectly fine and more readable.
- For repeated concatenation in loops → Use `StringBuilder`.

Key Points

- ✓ Both create new `String` objects.
- ✓ `concat()` works only with `Strings`.
- ✓ + can concatenate different data types.
- ✓ Prefer `StringBuilder` for repeated modifications.

Q64. When should you use `String`, `StringBuilder`, and `StringBuffer`?

Simple Answer

The choice depends on whether the String needs to be modified and whether thread safety is required.

- **String** → When the value does not change.
- **StringBuilder** → When frequent modifications occur in a single-threaded environment.
- **StringBuffer** → When frequent modifications occur in a multi-threaded environment.

Detailed Explanation

Use String

When data remains constant.

Examples:

- Employee ID
- PAN Number
- Country Name

Use StringBuilder

When repeatedly building or modifying text.

Examples:

- Report generation
- SQL query construction
- Log message creation

Use StringBuffer

When multiple threads modify the same text.

Examples:

- Legacy multi-threaded applications.
- Shared mutable character sequences.

Real-Time Example

Suppose a report generator appends thousands of records.

```
String report = "";
```

This creates many unnecessary String objects.

Instead:

```
StringBuilder report = new StringBuilder();
```

This improves performance significantly.

Code Example

```
StringBuilder builder = new StringBuilder();  
builder.append("Java");  
builder.append(" Spring Boot");  
builder.append(" Microservices");  
System.out.println(builder);
```

Output

```
Java Spring Boot Microservices
```

Interview Tip

A very common interview question:

Which class is used internally by the compiler for String concatenation?

Answer:

For simple concatenation, the Java compiler generally uses **StringBuilder** behind the scenes (except in certain compile-time constant expressions).

Key Points

- ✓ String → Immutable.
- ✓ StringBuilder → Fastest for modifications.
- ✓ StringBuffer → Thread-safe.
- ✓ Use the appropriate class based on the requirement.

Q65. What is the difference between a Process and a Thread?

Simple Answer

A **Process** is an independent program in execution with its own memory space.

A **Thread** is the smallest unit of execution within a process. Multiple threads can exist inside a single process and share the same memory.

Detailed Explanation

Process

- Independent execution.
- Own memory.
- Communication is expensive.
- Heavyweight.

Examples:

- Chrome
- VS Code
- Spotify

Each runs as a separate process.

Thread

A thread executes a task inside a process.

Threads share:

- Heap Memory
- Open Files
- Resources

Each thread has its own:

- Stack
- Program Counter

Threads are lightweight and faster than processes.

Comparison

Process	Thread
Independent	Part of a process

Own memory	Shared memory
Heavyweight	Lightweight
Slow creation	Fast creation

Real-Time Example

Google Chrome

Process

Chrome Browser

Threads

- UI Thread
- Download Thread
- Rendering Thread
- Network Thread

Code Example

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(Thread.currentThread().getName());  
    }  
}
```

Output

main

Interview Tip

A common interview question:

Why are threads faster than processes?

Answer:

Because threads share the same memory and require less overhead for creation and communication.

Key Points

- ✓ Process = Independent program.
- ✓ Thread = Smallest execution unit.
- ✓ Threads share Heap memory.
- ✓ Each thread has its own Stack.

Q66. What is Multithreading? What are its advantages?

Simple Answer

Multithreading is the ability of a program to execute multiple threads concurrently.

It improves application performance by allowing multiple tasks to run simultaneously.

Detailed Explanation

Instead of executing tasks one after another:

Task1

↓

Task2

↓

Task3

Threads execute concurrently:

Task1

Task2

Task3

Advantages:

- Better CPU utilization

- Faster execution
- Improved responsiveness
- Parallel task execution

Real-Time Example

Online Shopping Application

Different threads handle:

- Payment
- Email Notification
- Inventory Update
- Invoice Generation

All these tasks can execute concurrently.

Code Example

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
  
        System.out.println("Thread Running");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        MyThread thread = new MyThread();  
  
        thread.start();  
    }  
}
```

Interview Tip

Remember:

Always use

`start()`

instead of

`run ()`

Calling `run ()` directly executes it like a normal method without creating a new thread.

Key Points

- ✓ Executes multiple tasks concurrently.
- ✓ Improves performance.
- ✓ Better CPU utilization.
- ✓ Uses `start ()` to create a new thread.

Q67. What is the difference between extending the `Thread` class and implementing the `Runnable` interface?

Simple Answer

Both approaches are used to create threads.

However, implementing `Runnable` is preferred because it supports multiple inheritance and separates the task from the thread.

Detailed Explanation

Thread Class

```
class MyThread extends Thread
```

Advantages:

- Simple to implement.

Disadvantages:

- Java allows extending only one class.

Runnable Interface

```
class MyTask implements Runnable
```

Advantages:

- Supports multiple inheritance.
- Better object-oriented design.
- Reusable task implementation.

Comparison

Thread	Runnable
Extends Thread	Implements Runnable
Single inheritance limitation	No limitation
Less reusable	More reusable
Less preferred	Preferred

Real-Time Example

Spring Boot applications usually use:

Runnable

Callable

ExecutorService

instead of extending Thread.

Code Example

```
class MyTask implements Runnable {  
  
    @Override  
    public void run() {  
  
        System.out.println("Runnable Executed");  
    }  
}  
  
public class Main {
```

```
public static void main(String[] args) {  
    Thread thread = new Thread(new MyTask());  
    thread.start();  
}  
}
```

Interview Tip

Interview Question:

Which approach is recommended?

Answer:

Runnable, because Java supports only single inheritance.

Key Points

- ✓ Both create threads.
- ✓ Runnable is preferred.
- ✓ Better design.
- ✓ Supports multiple inheritance.

Q68. What is the Thread Lifecycle in Java?

Simple Answer

A thread passes through different states during its execution.

The lifecycle includes:

- New
- Runnable
- Running
- Waiting / Timed Waiting / Blocked
- Terminated

Detailed Explanation

New

Thread object is created.

```
new Thread();
```

Runnable

After calling:

```
start();
```

The thread becomes eligible for execution.

Running

CPU executes the thread.

Waiting / Blocked / Timed Waiting

Examples:

```
wait();
```

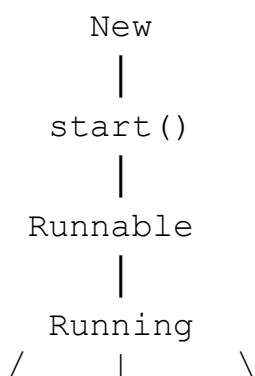
```
sleep();
```

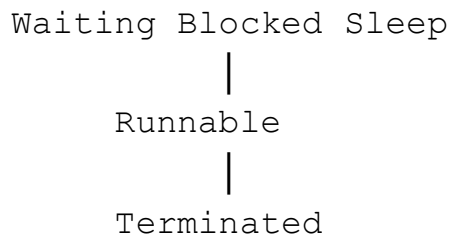
```
join();
```

Terminated

Thread completes execution.

Lifecycle Diagram





Real-Time Example

File Upload Application

- Thread created.
- Waits for CPU.
- Uploads file.
- Sleeps while waiting for network.
- Finishes execution.

Code Example

```
Thread thread = new Thread(() -> {  
  
    System.out.println("Running");  
});  
  
thread.start();
```

Interview Tip

One of the favorite interview questions:

Does calling `start()` immediately execute the thread?

Answer:

No.

It moves the thread to the **Runnable** state. The JVM scheduler decides when it actually starts running.

Key Points

- ✓ Thread starts in the New state.
- ✓ `start()` moves it to Runnable.

✓ Scheduler moves it to Running.

✓ Ends in the Terminated state.

Q69. What is Synchronization? Why is it needed?

Simple Answer

Synchronization is a mechanism in Java that ensures only one thread can access a shared resource at a time.

It prevents data inconsistency caused by multiple threads accessing the same resource simultaneously.

Detailed Explanation

When multiple threads modify the same object simultaneously, unexpected results may occur.

Synchronization ensures that:

- Only one thread enters the synchronized block or method at a time.
- Other threads wait until the current thread finishes.

Java provides synchronization using the `synchronized` keyword.

Real-Time Example

Consider a bank account.

Current Balance:

₹10,000

Two threads try to withdraw:

Thread A → ₹6,000

Thread B → ₹5,000

Without synchronization, both threads may read the same balance and allow withdrawals, resulting in an incorrect negative balance.

Synchronization ensures one withdrawal completes before the other begins.

Code Example

```
class BankAccount {  
    private int balance = 10000;  
    synchronized void withdraw(int amount) {  
        if (balance >= amount) {  
            balance -= amount;  
            System.out.println("Remaining Balance: " +  
balance);  
        }  
    }  
}
```

Interview Tip

A common interview question:

What keyword is used for synchronization?

Answer:

synchronized

Key Points

- ✓ Prevents data inconsistency.
- ✓ Ensures one thread accesses shared resources at a time.
- ✓ Uses the `synchronized` keyword.
- ✓ Important for thread safety.

Q70. What is a Deadlock? How can it be avoided?

Simple Answer

A **Deadlock** occurs when two or more threads wait indefinitely for each other to release resources.

As a result, none of the threads can continue execution.

Detailed Explanation

Example:

Thread A

Lock Resource A

↓

Waiting for Resource B

Thread B

Lock Resource B

↓

Waiting for Resource A

Neither thread can proceed.

Deadlock Diagram

Thread A -----> Resource A

↑

↓

Resource B <----- Thread B

Real-Time Example

Two employees each hold one key:

- Employee A has Key 1 and needs Key 2.
- Employee B has Key 2 and needs Key 1.

Neither employee can continue until the other releases the key.

Code Example

```
Object lock1 = new Object();  
Object lock2 = new Object();
```

Thread 1

```
synchronized(lock1) {  
  
    synchronized(lock2) {  
  
    }  
  
}
```

Thread 2

```
synchronized(lock2) {  
  
    synchronized(lock1) {  
  
    }  
  
}
```

This may lead to a deadlock.

How to Avoid Deadlock

- Acquire locks in a consistent order.
- Minimize nested synchronization.
- Use timeout-based locks (`ReentrantLock`).
- Reduce lock scope.

Interview Tip

Very common interview question:

Can Java automatically detect a deadlock?

Answer:

Yes.

Tools like **jstack**, **VisualVM**, and **Java Flight Recorder (JFR)** can help detect deadlocks during debugging.

Key Points

- ✓ Threads wait forever.
- ✓ Usually caused by inconsistent lock ordering.
- ✓ Can be prevented with proper locking strategies.
- ✓ Difficult to debug in production.

Q71. What is the `volatile` keyword?

Simple Answer

The `volatile` keyword ensures that changes made to a variable by one thread are immediately visible to other threads.

It provides **visibility**, but **not atomicity**.

Detailed Explanation

Without `volatile`:

Each thread may cache its own copy of a variable.

One thread may update the variable, but another thread might continue reading an outdated value.

With `volatile`:

Every read retrieves the latest value from main memory.

Real-Time Example

Suppose an application has a flag:

```
boolean applicationRunning;
```

One thread updates the flag:

false

Another thread continuously checks the value.

Declaring the flag as `volatile` ensures the second thread immediately sees the updated value.

Code Example

```
class Server {  
  
    volatile boolean running = true;  
  
    void stop() {  
        running = false;  
    }  
}
```

Interview Tip

One of the most frequently asked questions:

Does `volatile` make code thread-safe?

Answer:

No.

It guarantees **visibility**, but operations like `count++` are still not atomic.

Key Points

- ✓ Ensures visibility.
- ✓ Prevents stale values.
- ✓ Does not provide synchronization.
- ✓ Does not guarantee atomic operations.

Q72. What is the Executor Framework? Why is it preferred over creating threads manually?

Simple Answer

The **Executor Framework** is a high-level API introduced in Java 5 to simplify thread management.

Instead of creating and managing threads manually, tasks are submitted to an `ExecutorService`, which efficiently manages a pool of reusable threads.

Detailed Explanation

Without Executor Framework:

```
new Thread(...).start();
```

Creating many threads can lead to:

- High memory usage
- Context-switching overhead
- Poor scalability

With Executor Framework:

- Threads are reused.
- Thread pools improve performance.
- Code becomes cleaner and easier to maintain.

Real-Time Example

In an e-commerce application, thousands of customer requests arrive every minute.

Creating one thread per request is inefficient.

Instead, the application uses a thread pool managed by the Executor Framework to process requests efficiently.

Code Example

```
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) {

        ExecutorService executor =
        Executors.newFixedThreadPool(2);

        executor.submit(() ->
            System.out.println("Task Executed"));
    }
}
```

```
        executor.shutdown();  
    }  
}
```

Interview Tip

A common interview question:

Why is ExecutorService preferred over the Thread class?

Answer:

Because it reuses threads through thread pools, improving performance, scalability, and resource management.

Key Points

- ✓ Introduced in Java 5.
- ✓ Manages thread pools.
- ✓ Reuses threads efficiently.
- ✓ Preferred for enterprise applications.